

Универзитет у Београду
Електротехнички факултет
Катедра за електронику

Дигитална електроника 1

Лабораторијска вежба

Аритметичко логичка јединица (АЛУ)

Дизајн аритметичких и логичких кола за сабирање, одузимање и
поређење бројева

- други део -

аутори:

Марија Бежуљ

Никола Петровић

део материјала преузет са предмета Увод у пројектовање *VLSI* система

Београд, 2026.

Циљ вежбе

Циљ ове вежбе је да студенти генеришу VHDL фајлове за опис одговарајућег хардвера, упознају се са начином инстанцирања и коришћења компоненти, као и да науче да пишу VHDL код за тестирање и симулацију рада пројектованих кола.

Пројектовани систем треба да симулира рад једноставне Аритметичко Логичке Јединице (Arithmetic Logic Unit - *ALU*). На основу кода операције и датих 8-битних операнда *ALU* на свом излазу даје одговарајући резултат и сетује одговарајуће флегове. У овом делу упутства су описане компоненте које треба креирати у лабораторији.

Као и у првом делу, на располагању је веб апликација [EDA Playground](#) за писање кода и симулацију, док се *testbench* фајлови могу наћи на [линку](#).

Учитавање већ пројектованих фајлова

Отворити *EDA Playground* и одабрати playground у коме се налазе фајлови креирани према првом делу упутства. Преименовати playground у `lab1_gggg_bbbb` и подесити да је типа Public.

Напомена: У даљем делу лабораторијске вежбе се очекује да модули који су пројектовани код куће раде са обзиром да се наставак вежбе ослања на њих.

1 *MAX* модул

Пројектовати компоненту која одређује максимум два неозначена 8-битна броја, коришћењем претходно пројектованог 8-битног компаратора. На улазу се примају два осмобитна броја, а на излазу се прослеђује већи број. Предложена блок шема дата је на слици 1. Није неопходно пројектовати и инстанцирати мултиплексер $8 \times 2/1$ као независну компоненту.

Модул назвати `max_8` и креирати га у фајлу `max_8.vhd`.

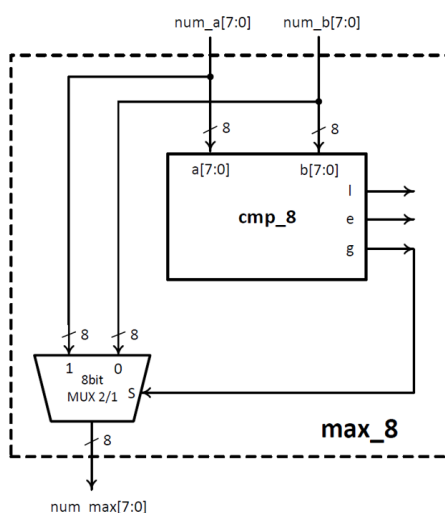
Покренути *testbench* фајл `max_8_tb.vhd` и испитати исправност рада *max* модула!

2 *MIN* модул

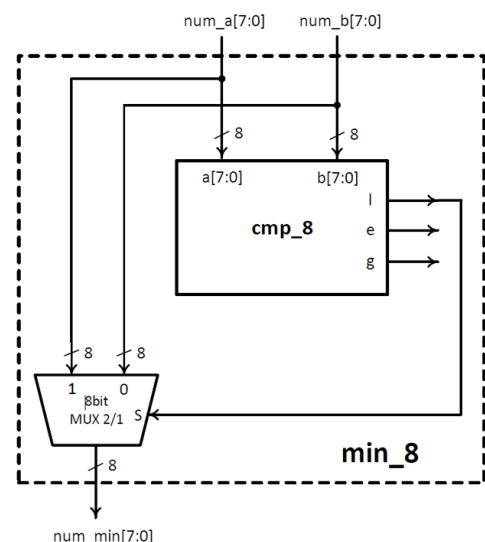
Пројектовати компоненту која одређује минимум два неозначена 8-битна броја, коришћењем претходно пројектованог 8-битног компаратора. На улазу се примају два осмобитна броја, а на излазу се прослеђује мањи број. Предложена блок шема дата је на слици 2. Није неопходно пројектовати и инстанцирати мултиплексер $8 \times 2/1$ као независну компоненту.

Модул назвати `min_8` и креирати га у фајлу `min_8.vhd`.

Покренути *testbench* фајл `min_8_tb.vhd` и испитати исправност рада *min* модула!



Слика 1: Блок шема *max* модула



Слика 2: Блок шема *min* модула

Напомена: Сигнали компоненте који нису повезани ни на један сигнал модула се при инстанцирању и писању *port map*-е "везују" на кључну реч *open*.

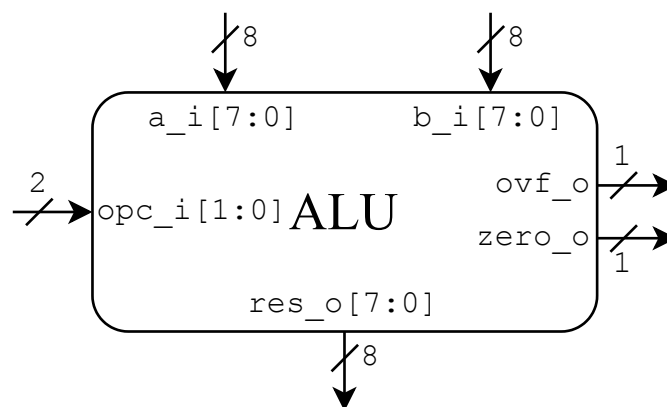
3 Пројектовање завршног модула - *ALU*

Користећи претходно пројектована аритметичко логичка кола, пројектовати модул *ALU* чији је интерфејс приказан на слици 3. Модул треба на основу операционог кода `opc_i[1..0]` и операнда `a_i[7..0]` и `b_i[7..0]` да генерише одговарајући излаз `res_o[7..0]` и постави флегове `ovf_o` и `zero_o`.

Операциони код дефинише операцију коју *ALU* извршава:

1. `opc_i[1..0] = 00` - максимум ($\max(a_i, b_i)$)
2. `opc_i[1..0] = 01` - минимум ($\min(a_i, b_i)$)
3. `opc_i[1..0] = 10` - сабирање ($a_i + b_i$)
4. `opc_i[1..0] = 11` - одузимање ($a_i - b_i$)

Флег `ovf_o` је активан ако је током извршене операције сабирања или одузимања дошло до прекорачења, за остале операције овај флег је једнак 0. Флег `zero_o` је активан уколико је резултат операције `res_o[7..0]` једнак 0.



Слика 3: Интерфејс модула *ALU*

Модул назвати *ALU* и креирати га у фајлу `ALU.vhd`.

Прекопирати *test-bench* фајл `ALU_tb.vhd` дат уз упутство и помоћу њега испитати исправност рада *ALU* модула.

Обавезно позвати дежурног асистента да потврди успешно одрађен овај део вежбе!